FightIQ Research Implementation Gameplan and Risk Register

Evidence-based sequencing for duration modeling, market comparison, and release safety

AUDITED REPOSITORY	C:\Users\nrmcn\predictor\threejs
GIT BASELINE	threejs-playground • commit 14bf2d4
AUDIT DATE	July 13, 2026
DOCUMENT STATUS	Standalone, implementation-verified baseline

EVIDENCE RULE Current behavior is separated from proposed, experimental, legacy, and uncertain material.

Document control and decision rules

This plan is derived from the implementation, generated artifacts, local data, experiment records, and current tests in the audited checkout. It is deliberately self-contained.

Evidence is ranked in this order:

1. Reproducible, leakage-safe experiments recorded in this repository.
2. Current local artifact metadata and database/file schemas.
3. Executable source code and tests.
4. Repository documentation that agrees with the implementation.
5. External or conceptual research claims that have not yet been reproduced here.

The label **Current** means implemented at the audited commit. **Proposed** means recommended work. **Experimental** means present but not production-serving. **Legacy/stale** means superseded or contradicted by newer repository evidence. Every numerical acceptance threshold in this document is marked as either an existing baseline or a proposed gate.

Table of contents

1. Executive decision summary

FightIQ should not treat the current `Model distance` percentage as a finished over/under model. **Current:** it is the broad method model's calibrated probability of the `Decision` class. That is useful context, but it does not estimate the probability that a fight lasts beyond a quoted sportsbook line such as 2.5 or 4.5 rounds. A fight can finish after the line and still settle Over, so `P(Decision)` systematically answers a different question.

The recommended sequence is:

19. Stabilize provenance, refresh observability, pipeline parity, and fighter identity boundaries.
20. Establish reliable totals-line ingestion and coverage reporting.
21. Build a dedicated calibrated duration/survival model that predicts `P(T > quoted threshold)`.
22. Ship it in shadow/admin mode, then as a clearly labeled model-versus-market comparison.
23. Accumulate prospective line-matched outcomes before displaying any edge or performance claim.
24. Pursue additional model research only through chronological, unique-fight, leakage-safe gates.

The six existing matchup interaction terms should not simply be wired into production. They were already evaluated across logistic regression, histogram gradient boosting, and XGBoost and were neutral to negative in recorded walk-forward experiments. They become worth revisiting only if new orthogonal inputs—such as reliable style annotations—change what the interactions represent.

Priority map

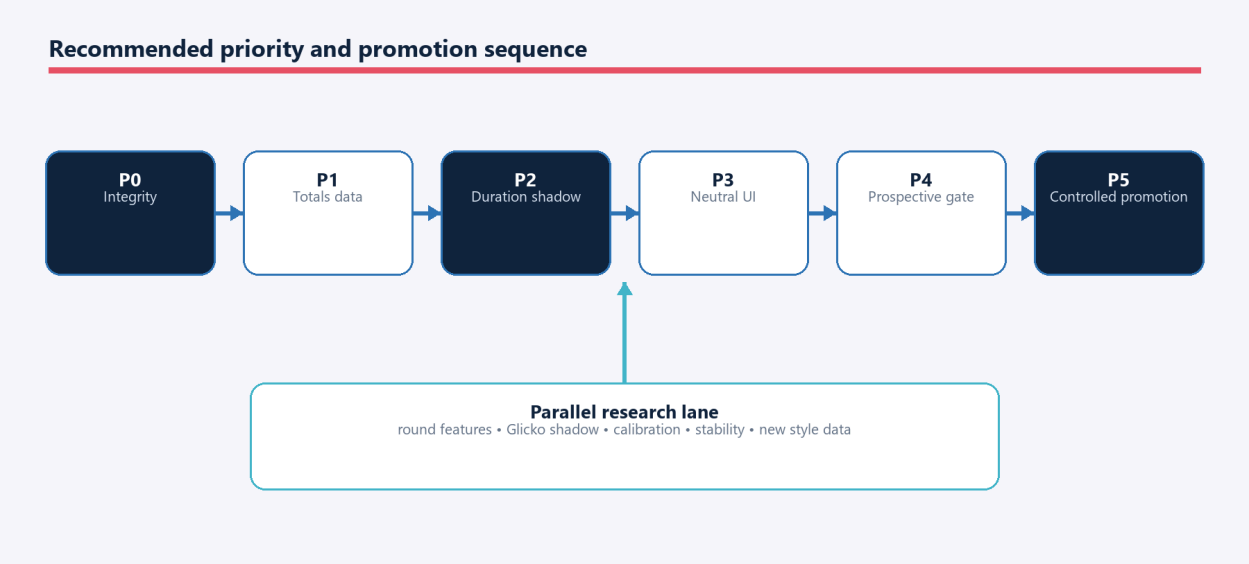

Figure 1. Recommended priority and promotion sequence

2. Verified baseline

2.1 Product and runtime

The application is a FastAPI backend plus React/Vite frontend. It supports winner predictions, method-derived context, upcoming cards, market odds, saved card snapshots, social picks, leaderboards,

evaluation views, fighter profiles, and an optional Three.js scene. SQLite is the live transactional store. CSV, JSON, and joblib artifacts support scraping, feature construction, training, and serving. Docker/Fly deployment packages a frontend build, backend, database, and selected model/data artifacts.

2.2 Model baseline

Item	Verified current state
Winner recipe	`backend/app/models/model_version.py`, version 1.2, recipe hash `d15652b351`
Serving model	Calibrated logistic winner model with 126 numeric inputs plus categorical `weight_class`
Latest recorded winner metrics	Accuracy 0.6315; Brier 0.2280; log loss 0.6490; AUC 0.6739
Chronological split	6,010 training fights; 859 calibration fights; 1,718 test fights
Training coverage	March 11, 1994 through July 11, 2026
Method model	Broad and detailed method models; 496 numeric inputs plus categorical `weight_class`
Current `Model distance`	Broad method model probability assigned to `Decision`
Provenance	Metrics include trained time, recipe hash, training-data hash, Git commit, and dirty-worktree flag

The latest winner artifact was trained from a dirty worktree. That does not prove the artifact is invalid, but it prevents the Git commit alone from reproducing the exact source state. Before evaluating new ideas, establish a clean and immutable comparison baseline.

2.3 Data and market baseline

Area	Verified current state	Planning consequence
Historical fights	8,772 `event_fights` rows in SQLite; exported raw CSV has 8,758	SQLite and compatibility CSVs can diverge; define authoritative reads
Training matchups	17,174 mirrored rows representing 8,587 unique fights	Split and score by unique fight, never by mirrored row
Current fighter features	2,694 rows and 134 columns	Duration work can reuse only pre-fight-safe columns
Upcoming schedule	8 events/56 fights in SQLite versus 6/51 in CSV	UI freshness must use SQLite or explicit regenerated exports
Moneyline odds	Implemented and persisted	Distinguish moneyline availability from totals availability
Totals fields	Schema and ingestion code exist	Implementation presence is not coverage
Totals coverage	0 of 58 current `future_fight_odds` rows have a non-null total	Duration UI must show unavailable state until data arrives
Odds history	24 SQLite rows versus 19 CSV rows	Use database-backed evaluation; exports are compatibility outputs

2.4 Validation baseline

At the audited state, local validation passed:

- Backend: 177 tests passed.
- Frontend: lint passed; 32 tests in 9 files passed; production build passed.
- Build warning: the lazy `OctagonScene` bundle is approximately 534 kB minified, above Vite's default 500 kB warning threshold.
- Backend warnings: FastAPI startup event deprecation and a pandas DataFrame fragmentation warning.

These results are a baseline, not proof that the unattended Windows scheduler, hosted upload, or real external odds and UFCStats integrations work continuously.

3. Research reconciliation

3.1 Findings that are ready to plan around

Finding	Evidence in this checkout	Decision
Duration should be modeled against the actual market threshold	Current UI compares a totals line with `P(Decision)`, which is not line-specific	Build a duration/survival target and retain `P(Decision)` only as separately labeled context if useful
Chronological validation is mandatory	Training code and existing tests recognize time order and mirrored fights	Use chronological, unique-fight splits for every experiment
Calibration matters as much as discrimination	The product exposes probabilities and market comparisons	Gate on Brier score, log loss, reliability curves, and subgroup calibration—not accuracy alone
Round-level history is most relevant to duration	`fight_round_stats.csv` exists with 40,474 rows	Build aggregated prior-round tendencies with strict pre-fight cutoffs
Prospective evaluation is needed	Saved model predictions and odds tracking already exist	Extend snapshots to line-specific duration predictions and settlements
Identity resolution is a platform dependency	Different name normalizers are used for prediction joins, odds, images, and ingestion	Introduce one canonical mapping boundary before broadening external data

3.2 Findings that conflict with repository experiments

Claim or idea	Stronger local evidence	Status and action
Six orphan matchup interactions are “free alpha” and should be wired	The exact terms were tested across logistic, histogram gradient boosting, and XGBoost; recorded deltas were neutral to negative	**Rejected for direct implementation.** Reopen only with new inputs or a materially different hypothesis
Method prediction is absent from Future Cards	`future_card_service.py` derives distance probability from the method model and `FutureCards.jsx` renders it	**Stale statement.** The value is present, but mislabeled for over/under use
Frontend has no automated tests and one fully eager bundle	Current suite has 32 tests and route-level lazy loading	**Historical state.** Add contract/E2E coverage; do not plan from the old count
A successful pipeline log proves daily automation is fixed	The latest log succeeded, but the chain depends on Task Scheduler, a Windows account, DPAPI, a local venv, network access, and upload auth	**Unproven operationally.** Add heartbeat and stale-data alerts

3.3 Ideas requiring new evidence

The following remain reasonable research directions, but none should be represented as an expected production gain:

- Glicko or uncertainty-aware rating features as a shadow alternative to current Elo.
- Round-level pace, damage, control, and durability aggregates for duration and method.
- Beta calibration, isotonic alternatives, or Venn–Abers-style intervals in a shadow comparison.
- Stability selection or regularization-path analysis performed inside training folds.
- Style clustering or hand-labeled archetypes, provided the labels are reproducible and available before the fight.
- Historical closing-line and scorecard enrichment, only from legally permitted, stable sources.

4. Target architecture

The duration capability should be a distinct, versioned model family rather than a rename of the method output.

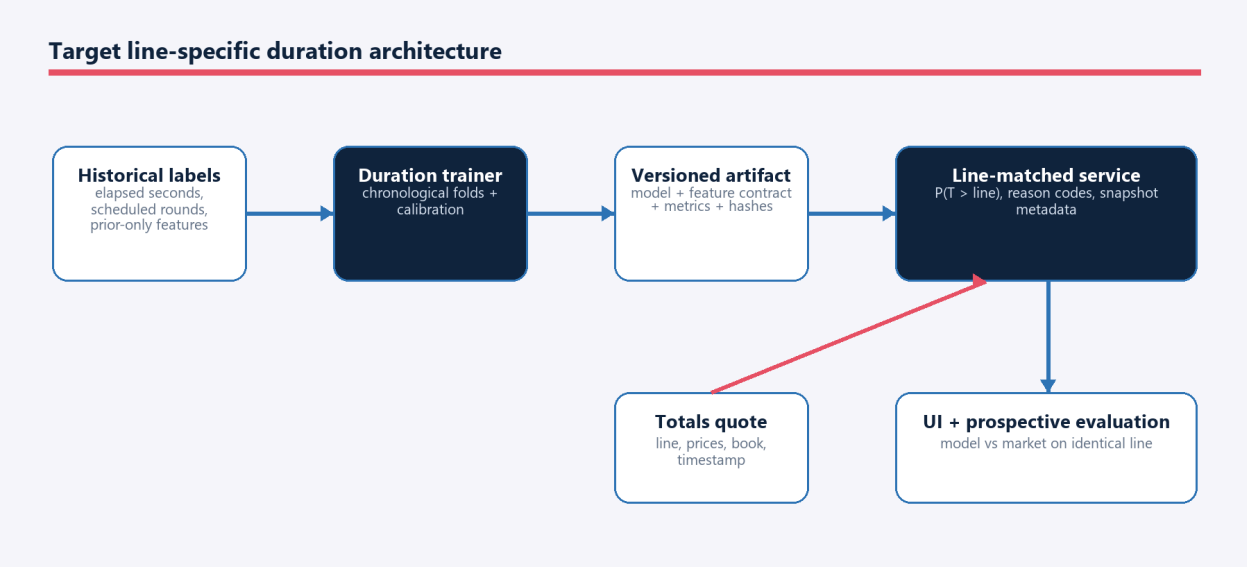

Figure 2. Target line-specific duration architecture

The service contract should preserve the distinction between three concepts:

- `decision_probability`: method model probability that the result is a decision.
- `over_probability`: duration model probability that elapsed time exceeds the quoted threshold.
- `market_over_probability`: de-vigged market probability for the same threshold and source timestamp.

No UI should subtract probabilities that refer to different events or different lines.

5. Phased implementation roadmap

Phase 0 — Integrity and observability foundation

Objective: make every later result reproducible and make “fresh” a measurable state.

Work item	Key files/components	Deliverable	Exit condition
Clean comparison baseline	`backend/app/training/train_model.py`; winner artifact metadata	A retrain from a clean commit or an immutable source diff captured in the manifest	Artifact can be reproduced or its exact non-Git inputs are archived
Artifact manifest	`deploy/make_bundle.py`; model registry; training scripts	JSON manifest with SHA-256, row counts, schemas, versions, Git state, and timestamps	Backend refuses or warns on incompatible/missing members
Shared pipeline registry	`backend/app/pipeline/incremental_pipeline.py`; `full_pipeline.py`	One declarative dependency list reused by both runners	Automated parity test proves required stages and ordering constraints
Refresh heartbeat	`backend/app/pipeline/auto_update.py`; admin settings/report; Data Ops UI	Last attempt, last success, failed stage, artifact age, upload result	Stale or failed refresh is visible without reading local logs
Authoritative-store policy	SQLite accessors and compatibility exporters	Explicit database ownership plus generated-export labels	Consumers no longer silently prefer stale CSVs
Identity boundary	ingestion, prediction, odds, image services	Canonical fighter ID/alias resolver with unresolved-match queue	Cross-source joins use one service and log match confidence

Rollback: these are additive controls. Keep legacy paths callable behind a temporary compatibility switch until manifest, pipeline, and resolver tests pass.

Phase 1 — Totals ingestion and coverage

Objective: prove that the application can receive, normalize, store, and display book totals for upcoming fights.

Actions:

25. Confirm the configured odds provider and account tier actually return the `totals` market for MMA.

26. Persist book, event, fighter mapping, line, Over/Under prices, source timestamp, fetch timestamp, and matching confidence.
27. Define aggregation behavior when books quote different lines. The current “most common line” may be retained initially, but the original per-book quotes must remain available.
28. Add coverage metrics by event and fight: moneyline available, totals available, books count, latest age, and unresolved identity matches.
29. Add fixture-driven tests for line disagreement, one-sided prices, pushes, missing outcomes, duplicate books, and stale quotes.
30. Render an explicit “Totals unavailable” state; never substitute `P(Decision)` for missing market data.

Exit condition: a representative upcoming-card fixture and at least one live authorized fetch populate line and both sides consistently; coverage is independently visible in Data Ops and Future Cards.

Phase 2 — Duration dataset and baseline model

Objective: build a leakage-safe baseline that answers the quoted-line question.

Actions:

31. Create a unique-fight duration table from completed-event date, scheduled rounds, official finish round/time, result status, weight class, bout type, and fighter identities.
32. Convert round/time into elapsed seconds with tested boundary rules.
33. Join only feature snapshots whose cutoff precedes the event.
34. Start with two baselines:

- calibrated binary classifiers for standard thresholds represented in available history; and
- a discrete-time hazard or survival model that estimates a full survival curve.

35. Evaluate both on identical chronological unique-fight folds.
36. Select the simplest approach that produces reliable line-specific calibration and supports current lines without proliferating fragile models.

Exit condition: the trainer produces a versioned model, feature contract, metrics report, lineage manifest, and reproducible out-of-fold predictions. No serving code changes are required to reach this gate.

Phase 3 — Shadow serving and snapshot evaluation

Objective: run the candidate without changing the public interpretation of Future Cards.

Actions:

- Load the duration artifact through the model registry and validate its schema/version.
- Calculate a prediction only when fighters, scheduled rounds, and a supported totals line are available.
- Store `line`, `over_probability`, `under_probability`, model version, artifact hash, source timestamp, feature-coverage flags, and prediction timestamp.
- Expose results first through an admin/debug response or Data Ops panel.
- Settle saved predictions after results refresh using the same boundary function used for training labels.

- Track coverage, Brier score, log loss, calibration, and confidence intervals prospectively.

Exit condition: shadow predictions survive at least one full event lifecycle from pre-fight snapshot through result settlement with no manual database edits.

Phase 4 — Future Cards presentation

Objective: show a line-matched comparison without overstating certainty.

Recommended row:

Display element	Example	Rule
Book line	Over/Under 2.5 rounds	Show source/book count and quote age
Market	Over 58% / Under 42%	De-vig both sides from the same line
Model	Over 54% / Under 46%	Same line; show model version in details
Difference	Model − market: −4 pts on Over	Neutral comparison language initially
Context	Decision probability 48%	Optional, separately labeled; never called the 2.5-round probability
Availability	Duration model unavailable	Give a reason category without exposing sensitive internals

Do not label a side as a bet, edge, or recommendation until the prospective evaluation gate is met and product language is explicitly approved.

Exit condition: API, frontend mock fixtures, UI tests, responsive states, saved snapshots, and interpretation documentation agree on the same field definitions.

Phase 5 — Prospective gate and controlled promotion

Objective: determine whether line-specific estimates are useful and stable enough for stronger presentation.

Promote only when:

- every scored prediction was saved before the relevant fight and market close;
- market and model refer to the identical line;
- settlement rules are verified against official results and target-book rules;
- coverage and exclusions are published with results;
- confidence intervals do not support a material regression versus the declared baseline;
- subgroup calibration is acceptable for 3-round/5-round fights and high-volume weight classes;
- no training/serving feature mismatch or identity error was observed in the evaluation set.

Proposed sample gate: review after 200–300 line-matched, prospectively saved fights, but do not treat the number alone as sufficient. This threshold is a planning proposal, not an established statistical guarantee; power should be calculated from observed event rates and the minimum useful improvement.

6. Fight-duration and totals implementation specification

6.1 Target definition

Let `T` be official elapsed fight time in seconds and `L` be the sportsbook threshold converted to seconds. The primary prediction target is:

`over = 1 if T > L, otherwise 0`, subject to the selected book's void/push rules.

Examples under the commonly used half-round interpretation:

Market line	Candidate threshold conversion	Example outcome
1.5 rounds	450 seconds	Finish at 2:31 of Round 2 has elapsed 451 seconds and is Over
2.5 rounds	750 seconds	Finish at 2:31 of Round 3 has elapsed 751 seconds and is Over
4.5 rounds	1,350 seconds	Finish at 2:31 of Round 5 has elapsed 1,351 seconds and is Over

Implementation caution: the exact equality boundary, cancellations, technical decisions, no contests, overturned results, and unusual round lengths must be verified against the rules of the market source being displayed. Encode them once in a settlement module and use the same function for labels, API grading, and tests.

6.2 Candidate model form

Preferred first comparison:

- Baseline A: calibrated logistic model per supported threshold or with line as an explicit input.
- Candidate B: discrete-time hazard model with intervals within rounds, from which `P(T > L)` is read directly.

The survival form is attractive because one model can produce coherent probabilities across 1.5, 2.5, and 4.5 lines and can expose the full duration curve. It is not automatically superior. Promote it only if calibration and stability beat the simpler baseline under identical folds.

Avoid training only a “goes to decision” classifier. That target collapses all late finishes into the Under side for every line and cannot support arbitrary thresholds.

6.3 Feature policy

Candidate features must be known before the fight and calculated from prior bouts only:

Feature family	Examples	Primary safeguard
Fight context	Scheduled rounds, weight class, title/main-event indicators if reliably encoded	Schema and value-domain tests
Activity and experience	Prior UFC bouts, days since last fight, recent bout count	Event-date cutoff tests
Prior duration	Mean/median elapsed time, decision rate, early/late finish rates	Minimum-sample shrinkage; no current-fight result
Pace and output	Prior attempts/landed per minute, position-specific activity	Denominator and zero-time tests

Durability	Prior knockdowns absorbed, stoppage history, damage-rate proxies	Missingness and sparse-sample flags
Control/grappling	Prior control share, takedown/submission attempts	Round aggregation verified against source schema
Matchup differences	Symmetric fighter A/B differences and sums	Red/blue swap invariance test
Market line	Threshold seconds or round fraction	Never use post-close outcome or closing price in a pre-open model

Do not import the existing six interaction terms by default. If new style features justify new interactions, define the hypothesis before looking at the final holdout and test them as a bounded ablation.

6.4 Dataset contract

Proposed files/modules:

- `backend/app/features/duration_dataset.py`: unique-fight label creation and prior-only feature assembly.
- `backend/app/training/train_duration_model.py`: chronological train/calibration/test procedure.
- `backend/app/training/evaluate_duration_model.py`: line and subgroup metrics plus reliability artifacts.
- `backend/app/models/duration_model.py`: artifact loader and compatibility checks.
- `backend/app/services/duration_service.py`: line-specific inference and reason-coded unavailable states.
- `backend/app/services/totals_settlement.py`: one tested source of truth for threshold conversion and grading.

The exact module names are proposed. Reuse existing generic helpers where doing so does not blur winner, method, and duration contracts.

One row should represent one fight/line evaluation unit. If multiple lines per historical fight are synthesized for training, group every derived row from the fight into the same fold and report performance separately by actual-versus-synthetic line availability.

6.5 Artifact contract

Proposed artifact family:

Artifact	Required contents
`duration_model.joblib`	Fitted calibrated pipeline or survival estimator
`duration_model_features.json`	Ordered numeric features, categorical features, definitions, null/default policy, schema version
`duration_model_metrics.json`	Splits, metrics, reliability summaries, subgroup counts, exclusions, lineage
`duration_model_registry.json` entry	Model version, relative paths, hashes, status, compatibility versions
Out-of-fold predictions	Fight ID, event date, line, target, prediction, fold, model version; no secrets

The serving process must reject an artifact when its feature-schema version, ordered feature list, or preprocessing version does not match the serving builder.

6.6 API contract

Proposed response fragment:

```
{
  "duration_prediction": {
    "line": 2.5,
    "line_seconds": 750,
    "over_probability": 0.54,
    "under_probability": 0.46,
    "model_version": "duration-1.0.0",
    "calibrated": true,
    "coverage": "full",
    "prediction_timestamp": "2026-07-13T18:00:00Z"
  }
}
```

When unavailable, return a stable reason code such as `totals_line_missing`, `fighter_unresolved`, `unsupported_round_format`, `feature_row_missing`, or `artifact_unavailable`. Do not manufacture a percentage from the method model.

During migration, keep `model_distance_probability` only as a deprecated, explicitly defined decision probability. Remove it after all consumers and saved snapshots have migrated.

6.7 Tests required before display

- Finish-time conversion for every round and exact half-round boundaries.
- Five-round, three-round, one-round anomaly, no-contest, draw, overturned, and missing-time fixtures.
- Mirrored-row and chronological group isolation.
- Training/serving feature-name, order, dtype, category, and null-policy equality.
- Red/blue swap invariance or documented complement behavior.
- Calibration and metric calculation on fixed fixtures.
- API contract, unavailable reason codes, frontend mock parity, and UI responsive states.
- Snapshot immutability, settlement idempotence, and no post-fight overwrite.
- Multiple books/lines, de-vigging, quote age, and exact line matching.
- Artifact incompatibility and rollback behavior.

7. Model and data research tracks

These tracks can begin only after Phase 0 establishes reproducible baselines. They should not compete for the same untouched final holdout.

7.1 Round-level duration features — highest modeling priority

Use `backend/data/processed/fight_round_stats.csv` to create prior-only aggregates such as early-round output, pace decay, control share, knockdown exposure, and finish timing. Every aggregate needs

a cutoff test proving that fight `N` does not use round data from fight `N` or later. Add empirical-Bayes shrinkage or explicit sample-size features so a one-fight athlete is not treated as precisely estimated.

Why first: these variables directly describe the process that determines duration. They are more causally aligned with totals than generic winner interactions.

7.2 Glicko or uncertainty-aware ratings — shadow comparison

Implement a rating provider behind the same feature interface as current Elo. Produce pre-fight rating, rating deviation/uncertainty, and matchup differences. Compare:

- current Elo only;
- Glicko-style features only; and
- both, with regularization.

Gate on chronological out-of-fold log loss and Brier score for winner prediction, plus calibration by experience. Do not replace Elo merely because the new rating is theoretically richer.

7.3 Calibration alternatives

Compare the current sigmoid calibration against carefully nested alternatives. Calibration fitting must occur without touching the final test period. Evaluate overall and subgroup reliability, expected calibration error with fixed/adaptive bins, Brier decomposition where practical, and bootstrap confidence intervals. A method that creates sharper-looking probabilities but worse log loss or unstable tails should remain experimental.

7.4 Feature stability and selection

Measure coefficient/sign stability and feature selection frequency across chronological folds. Any filtering, imputation choice, or hyperparameter selection must occur within the training portion of each fold. Use this to simplify feature families and identify brittle inputs, not to mine the final holdout repeatedly.

7.5 Style representation and new interactions

If style labels are added, first define how they are produced, versioned, and available before each historical fight. Candidate sources include reproducible clustering from prior-only round features or human-reviewed labels with inter-rater checks. Only then test bounded interactions such as pace-versus-durability or wrestling-pressure-versus-takedown-defense. This is the condition under which revisiting interaction hypotheses becomes meaningfully new research.

7.6 Historical odds and scorecards

Before ingestion, verify terms of use, licensing, update stability, identity coverage, and historical timestamp semantics. Closing odds cannot be used as an input to a model presented as an earlier pre-fight forecast. Scorecards can support judging and round-outcome research but have sparse and selection-biased availability; keep them out of production features until coverage and missingness are documented.

8. Reliability and release engineering

8.1 Daily refresh

Current: Windows Task Scheduler launches `backend/app/pipeline/auto_update.ps1`, which decrypts a DPAPI-protected credential, runs `auto_update.py`, executes the incremental pipeline, builds a bundle, and uploads it to the admin endpoint. The newest inspected log completed successfully, including the incremental update and hosted upload path. However, a successful log does not prove the job runs every day.

Required improvements:

- Persist `last_attempt_at`, `last_success_at`, `last_failure_stage`, `duration`, `local_bundle_hash`, and `upload_result` in a status record exposed to Data Ops.
- Add a stale threshold independent of the scheduler; alert when the latest successful data date or run age exceeds policy.
- Record explicit `skipped` reasons for optional odds and image stages. A best-effort skip is not equivalent to refreshed data.
- Add a post-upload hosted health check that verifies manifest hash, database row counts, model versions, and latest event date.
- Move the runner to an always-on CI/hosted system when credentials, scraping constraints, and cost permit. Until then, document that a sleeping/offline Windows machine cannot guarantee wall-clock daily execution.

8.2 Atomic artifact deployment

Current risk: a bundle upload replaces files sequentially. Requests can observe a new model with old features or a new database with old metadata.

Target flow:

37. Upload into a versioned staging directory.
38. Validate archive paths, manifest, checksums, schema compatibility, model load, and a smoke prediction.
39. Quiesce or use a generation pointer.
40. Switch the whole artifact generation atomically.
41. Retain the previous generation for one-command rollback.
42. Report the active generation/hash through `/health` or an admin status endpoint.

8.3 Typed contracts

Many FastAPI routes return untyped dictionaries and the frontend API client has handwritten assumptions. Introduce Pydantic response models for high-change endpoints, generate or validate frontend types from OpenAPI, and run a real-backend contract test in CI. Keep mocks as scenario fixtures, not as the definition of the production contract.

8.4 Database and schema evolution

The database currently relies on `PRAGMA user_version=1` plus forward-compatible column additions. Add numbered, idempotent migrations and a compatibility matrix for the backend, bundle, and database

schema. Duration snapshots and settlements require migrations that preserve old rows and identify which model/line semantics produced them.

9. Measurement and acceptance gates

9.1 Universal experiment gate

Every candidate must declare before the final evaluation:

- hypothesis and expected mechanism;
- primary metric and minimum useful change;
- unit of independence: unique fight;
- chronological train, calibration, and untouched test windows;
- feature availability timestamp;
- comparison baseline and identical eligible rows;
- subgroup and missingness plan;
- artifact and code provenance;
- promotion, rejection, and rollback rule.

9.2 Duration metrics

Metric	Purpose	Gate principle
Brier score	Probability accuracy	No material regression against declared baseline on line-matched fights
Log loss	Penalizes confident errors	Primary paired comparison when probabilities are the product
Calibration curve/intercept/slope	Reliability	No systematic overconfidence; inspect 3- and 5-round subgroups
ROC AUC	Ranking	Secondary only; good AUC cannot rescue poor calibration
Coverage	Operational usefulness	Report denominator and every exclusion reason
Line agreement	Contract integrity	100% of scored comparisons use exactly matching model and market lines
Prospective result	Real-world stability	Only immutable pre-fight snapshots count

Existing winner-model values in Section 2 are comparison baselines for winner work, not duration acceptance thresholds.

9.3 Proposed promotion rule

Use paired bootstrap confidence intervals on identical fights. Promote a duration candidate only if it improves or is statistically compatible with the simpler baseline on the primary proper-scoring rule, does not materially degrade calibration or key subgroups, and adds enough coverage/maintainability value to justify complexity. Define “material” before the final test after a pilot power calculation; do not invent a favorable threshold after viewing results.

For the UI, separate technical promotion from stronger product language. A model can be technically sound enough to display as context before it is proven to identify market value.

10. Risk and concern register

ID	Severity	Concern	Likely impact	Mitigation / decision
R1	Critical	`Model distance` answers `P(Decision)`, not `P(Over line)`	Misleading market comparison and false edge interpretation	Separate concepts; dedicated duration target; migrate API/UI labels
R2	Critical	Current totals coverage is 0 of 58 rows	Feature can appear implemented while providing no usable comparisons	Coverage telemetry, live-provider verification, unavailable state
R3	Critical	Model artifacts and most generated data are ignored and the current winner artifact records a dirty worktree	Irreproducible baselines and accidental artifact drift	Immutable manifest, hashes, clean-source requirement or archived diff
R4	Critical	Bundle members are replaced sequentially	Mixed incompatible generations during requests	Stage, validate, and atomically switch versioned generations
R5	High	Full and incremental pipelines duplicate stage lists and ordering	A fix can update one path but not the other	Shared declarative stage registry and parity tests
R6	High	Name normalization differs across prediction, odds, images, and ingestion	Wrong or missing fighter joins; silent odds mismatch	Stable fighter IDs, alias registry, confidence and review queue
R7	High	Scheduler depends on a local Windows session, DPAPI, venv, connectivity, and machine availability	Missed daily refresh with no immediate visible failure	Heartbeat, stale alert, post-upload check, eventual always-on runner
R8	High	Current SQLite and compatibility CSV row counts diverge	Training, UI, and debugging can use different “current” data	Define ownership; generate exports with timestamps/manifests
R9	High	Duration labels have bookmaker-specific boundary and void rules	Wrong training targets and grading at exact boundaries	One versioned settlement module with provider-specific fixtures
R10	High	Mirrored matchup rows can cross validation folds	Leakage and exaggerated model performance	Group by canonical fight ID; assert fold isolation
R11	High	Retrospective model snapshots can be overwritten or created after outcomes	Inflated prospective evaluation	Immutable timestamped snapshots and idempotent settlement
R12	High	Repository prose recommends interactions contradicted by recorded experiments	Repeating failed work and holdout mining	Mark rejected result and require a new hypothesis/input
R13	Medium	FastAPI response dictionaries and handwritten frontend client are weak contracts	Runtime UI breakage after API edits	Pydantic responses, generated/validated types, real-API CI
R14	Medium	Market aggregation can collapse books with different totals lines	Invalid average and model-market comparison	Retain per-book quotes; aggregate only identical lines or state method
R15	Medium	Odds, scorecard, or image sources may have licensing/ToS constraints	Forced removal, blocked scraper, or legal exposure	Source review and documented permitted use before expansion
R16	Medium	Sparse fighters and missing round history create unstable duration estimates	Confident-looking probabilities from thin data	Shrinkage, sample-size features, coverage flags, subgroup calibration

R17	Medium	Large backend services and frontend views concentrate responsibilities	Risky changes and slow review	Extract duration/settlement/contract modules behind tests
R18	Medium	SQLite migration mechanism is minimal	Hosted upgrade or rollback failures	Numbered migrations and tested upgrade/downgrade policy
R19	Medium	Tracked generated odds JSON can leave routine working trees dirty	Accidental commits and ambiguous provenance	Decide fixture vs live artifact; move live copy to ignored storage
R20	Medium	About 127 MB of tracked UI-review material and duplicate snapshots	Repository bloat and confusing “current” UI evidence	Archive/remove superseded assets and retain a curated latest set
R21	Low	Three.js chunk exceeds the default size warning	Slower first load when scene is opened	Profile, split dependencies/assets, set a measured budget
R22	Low	Deprecated FastAPI startup hook and pandas fragmentation warning	Future framework break and avoidable performance cost	Move to lifespan API; assemble feature columns in batches

11. Dependencies, sequencing, and rollback

11.1 Dependency matrix

Workstream	Must precede it	Downstream unlocks	Rollback unit
Manifest and clean baseline	None	Trustworthy experiments and deploys	Previous artifact generation
Shared pipeline/refresh status	Manifest recommended	Reliable data and prospective tracking	Legacy runner plus status fields ignored
Canonical identity	Schema/migration plan	Odds, images, cross-source research	Alias resolver compatibility mode
Totals ingestion	Identity and coverage telemetry	Duration market comparison	Hide totals fields; retain moneyline
Duration dataset/model	Clean baseline; settlement rules	Shadow duration service	Disable duration registry entry
Snapshot settlement	Duration API contract; DB migration	Prospective evaluation	Preserve rows, stop new writes
Public UI	Shadow lifecycle and tests	User-facing model comparison	Feature flag or response omission
Edge language	Prospective sample and review	Stronger product positioning	Return to neutral comparison wording

11.2 Suggested working increments

43. One pull request: manifest, source-state capture, `/health` generation fields, tests.
44. One pull request: shared pipeline registry and scheduler heartbeat/status UI.
45. One pull request: canonical identity service adopted first by odds ingestion.
46. One pull request: totals fixtures, persistence, coverage, and unavailable UI—no duration model yet.

47. One experimental branch: duration labels, baselines, chronological evaluation; no serving changes.
48. One pull request: selected duration artifact contract and shadow serving.
49. One pull request: snapshot settlement and evaluation.
50. One pull request: neutral public presentation behind a feature flag.

Small, reversible increments make it clear which change caused any data, calibration, or deployment regression.

11.3 Stop conditions

Pause the duration rollout when:

- totals cannot be obtained reliably or permissibly;
- settlement semantics cannot be verified for the displayed source;
- canonical fighter matching produces unresolved or ambiguous high-profile bouts;
- training and serving features cannot be shown identical;
- artifact provenance is incomplete;
- shadow results reveal calibration failure or line mismatches;
- deployment cannot guarantee a coherent artifact generation.

12. Recommended next work session

The best next work is a short P0 foundation slice followed immediately by the duration dataset—not another broad model experiment.

Recommended scope:

51. Add an artifact-manifest schema and generation command.
52. Record refresh last-attempt/last-success/failure/upload fields and expose them in Data Ops.
53. Add totals coverage diagnostics and fixture-driven tests.
54. Define and test the official elapsed-time/line settlement function.
55. Produce a duration-dataset audit report with row counts, exclusions, target rates by line, rounds, era, and weight class.

At that checkpoint, review the audit before choosing logistic-per-line versus survival modeling. This sequence directly advances the improved Future Cards duration feature while reducing the chance of building it on stale data or ambiguous labels.

Alternative if operational reliability is the urgent concern: complete items 1–3 and run the scheduler through a full local-to-hosted cycle before any model work. Alternative if research is the priority: items 1 and 4–5 are the minimum safe prerequisite.

13. Practical kickoff checklist

- Confirm the working tree and record unrelated changes before editing.
- Confirm the active database, model registry, and artifact generation.
- Create or verify a clean reproducible winner baseline; do not compare against an unexplained dirty artifact.

- Choose the totals provider/source and verify permitted access and exact settlement rules.
- Add fixtures before relying on live external responses.
- Define the canonical fight ID and fighter identity mapping used by labels, odds, snapshots, and results.
- Write the finish-time conversion and boundary tests before generating duration labels.
- Audit unique fights, exclusions, missing values, era coverage, scheduled rounds, and target balance.
- Freeze the chronological split and untouched final test period.
- Declare primary metric, baseline, minimum useful change, and promotion rule.
- Store out-of-fold predictions and full provenance for every candidate.
- Keep duration artifacts separate from winner and method artifact families.
- Run backend tests, frontend lint/tests/build, API contract checks, and a real-data smoke test.
- Exercise a full pre-fight snapshot through result settlement before public display.
- Ship neutral model-versus-market wording first; keep `P(Decision)` separately labeled.
- Maintain a feature flag and previous coherent artifact generation for rollback.